

SERVIDOR HTTP

PYTHON



6 DE OCTUBRE DE 2026

YAZMIN RODRIGUEZ SALAZAR

El propósito de esta práctica fue construir un servidor HTTP completo utilizando exclusivamente la biblioteca estándar de Python (http.server), implementando una arquitectura CRUD (Create, Read, Update, Delete) para administrar una bitácora de incidencias de seguridad. Como requisitos clave, el sistema no emplea bases de datos relacionales, sino persistencia basada en un archivo de texto en formato JSON (incidencias.json), incorporando además un sistema interno de auditoría mediante contadores y bitácora de peticiones.

2. Arquitectura y Componentes del Servidor

El desarrollo se estructuró en torno a los siguientes componentes lógicos:

- **Herencia y Manejo de Peticiones:** La clase Manejador hereda de http.server.BaseHTTPRequestHandler para escuchar en el puerto 8001, sobrescribiendo los métodos HTTP estándar (GET, POST, PATCH, PUT, DELETE y HEAD).
- **Persistencia (cargar y guardar):** Las funciones auxiliares leen el archivo incidencias.json al arrancar el servidor y calculan automáticamente el siguiente identificador (siguiente_id = max(i["id"]) + 1) para evitar duplicidad de folios tras reinicios. Cada modificación reescribe el archivo preservando acentos (ensure_ascii=False) y aplicando sangrías legibles (indent=2).
- **Auditoría Centralizada (_enviar y _leer_json):** Se implementó una función unificada de respuesta que empaqueta diccionarios a JSON, calcula los encabezados de tamaño (Content-Length) y registra automáticamente cada transacción en las estructuras volátiles de logs y contadores.

3. Desarrollo y Explicación de los Ejercicios

Vocabulario controlado

- **Explicación:** Se estableció una tupla de estados permitidos: ESTADOS_VALIDOS = ("abierta", "en_proceso", "resuelta").
- **Funcionamiento:** En los métodos do_POST y do_PATCH, el servidor evalúa si el campo estado provisto en el cuerpo de la petición pertenece a esta tupla. Si un usuario intenta registrar o actualizar un estado inválido (por ejemplo, "cerrado" o valores arbitrarios), el servidor intercepta la acción y responde inmediatamente con un código 400 Bad Request, devolviendo un mensaje de error que enumera explícitamente los valores válidos aceptados.

La libreta en disco vs. Memoria RAM

- **¿Por qué las incidencias persisten al reiniciar el servidor?** Porque cada operación de escritura (POST, PATCH, PUT, DELETE) invoca explícitamente a la función guardar(), la cual serializa la lista de Python en el archivo físico incidencias.json mediante json.dump(). Al reiniciar el script,

la función cargar() lee dicho archivo del disco y restaura tanto los registros como el contador de folios.

- **¿Por qué /logs y /contadores vuelven a cero al reiniciar?** Porque las estructuras logs y contadores están declaradas exclusivamente como variables globales en la memoria RAM del proceso de Python. No se escriben en ningún archivo secundario de disco. Al detener el servidor (Ctrl+C) y volver a iniciarlo, el espacio de memoria se libera por completo y las estadísticas de la sesión anterior se reinician.

El 501 invisible

- **Explicación:** Si un cliente ejecuta una petición con un método no soportado (como PUT antes de ser programado), la clase base BaseHTTPRequestHandler detecta la carencia del método y responde automáticamente con un código 501 Not Implemented.
- **¿Por qué no aparece en /contadores ni en /logs?** Porque esa respuesta es generada de forma interna e inmediata por la clase padre de la biblioteca estándar antes de que el flujo de ejecución alcance nuestros métodos personalizados. Dado que dicha respuesta no pasa por nuestra función centralizada _enviar(), el sistema no tiene la oportunidad de registrar la petición en la lista de logs ni de incrementar las estadísticas en el diccionario de contadores.

Implementación de do_PUT

- **Explicación:** Se implementó el método do_PUT dentro de la clase Manejador para realizar un reemplazo completo del registro.
- **Funcionamiento:**
  - Extrae y valida que el identificador en la ruta sea un número entero.
  - Lee el cuerpo de la petición asegurando que contenga obligatoriamente los campos tipo y descripcion (de lo contrario, responde 400 Bad Request).
  - Localiza el registro en la lista, sustituyendo sus atributos pero **conservando inalterado su identificador original** (id).
  - Reinicia por defecto el campo estado a "abierta", actualiza la marca de tiempo, invoca guardar() en disco y responde con código 200 OK enviando el objeto actualizado.

Filtros combinados en GET /incidencias

- **Explicación:** Se actualizó la rama correspondiente a la ruta /incidencias dentro de do_GET utilizando parse_qs.
- **Funcionamiento:** El servidor analiza los parámetros de consulta (query string) en la URL y evalúa de manera independiente y acumulativa los filtros de tipo y estado (por ejemplo: /incidencias?tipo=alerta&estado=abierta). Esto permite realizar consultas precisas combinando ambos criterios tanto

desde clientes automatizados en Python como mediante comandos directos con curl.

4. Conclusiones

La práctica permitió comprender a bajo nivel el funcionamiento del protocolo HTTP y la construcción de APIs REST sin depender de frameworks externos pesados. El uso de la biblioteca estándar de Python demostró cómo gestionar cabeceras, códigos de estado y serialización JSON, mientras que la persistencia en archivos planos y la auditoría de eventos consolidaron los fundamentos de la ingeniería de software aplicada a la ciberseguridad y el diseño de sistemas distribuidos.